RepairDesk — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and where each lives

Keep one job per file. A clean layout:

- `src/config.py`, `src/exceptions.py`, `src/utils/logger.py` — settings, one named error (`MissingConfigError`), and one logging setup. (Doc01's pattern — reuse it.)
- `src/models/llm.py` — the **only** file that talks to OpenAI: `create_client(config)`, a one-token `check_api_key_at_startup(client)`, `send_message()` and `stream_message()`.
- `src/schemas.py` — the two fixed shapes: `RepairTicket` and `RouteDecision` (Pydantic `BaseModel`).
- `src/agents/intake.py`, `concierge.py`, `router.py` — one agent each.
- `src/services/shop_service.py` — `handle_message()` (router → agent) and `print_ticket()`.
- `src/utils/history.py` — `estimate_tokens()` and `trim_history()` (memory stays under the limit).
- `src/prompts/*.txt` — one prompt per agent, in plain text.
- `main.py` — the terminal loop.

Build order that works well (but it's your call): a chat that replies → add memory + streaming → add the `RepairTicket` shape and the Intake agent → add the Router and tie it together in `handle_message()`.

Hint 2 — the strict ticket, and the model that fills it

Make the ticket a shape the model **cannot** break:

```
from typing import Literal
from pydantic import BaseModel

class RepairTicket(BaseModel):
    customer_name: str | None            # None = the customer didn't say
    device: Literal["phone", "laptop", "tablet", "other"]
```

```
    urgency: Literal["low", "medium", "high"]
    summary: str
```

- `str | None` is what lets the name be **empty** — so the agent never has to invent one. Say so in the prompt too: *"fill customer_name only if the customer states their name — otherwise leave it null, never guess."*
- `Literal[...]` stops the model returning free text like `"Phone (cracked)"` for the device.

To make the model answer in that exact shape, use the SDK's structured parse (not plain text you parse yourself):

```
completion = client.beta.chat.completions.parse(
    model=MODEL, messages=messages, response_format=RepairTicket,
)
reply = completion.choices[0].message
if reply.refusal:        # the model declined
    return None
return reply.parsed      # a real RepairTicket object
```

Wrap it in `try/except ValidationError` and return `None` on a bad answer — never pass junk on.

Hint 3 — the router, with a safe fallback

`RouteDecision` is the same idea — a tiny fixed shape:

```
class RouteDecision(BaseModel):
    agent: Literal["concierge", "intake"]
    reason: str
```

`decide(client, message)` asks the model (same `parse` call, `response_format=RouteDecision`) which agent should handle the message. **Start with a safe default** and only replace it if the parse works:

```
decision = RouteDecision(agent="concierge", reason="router could not decide")
# ... try the parse; if reply.parsed is not None: decision = reply.parsed
```

So an unclear or failed route still answers (as the Concierge) instead of crashing. Log the decision and its reason.

Hint 4 — the chat, and never crashing

- **Memory** is just a `history` list you keep adding to and resend every call. The model remembers nothing on its own.
- **Streaming**: pass `stream=True`, loop the chunks, `print(piece, end="", flush=True)` — the `flush=True` is what makes it appear live — and join the pieces into the full reply.
- **Trim before every call** (`trim_history`): drop the oldest turns but **never** the system prompt (index 0), so a long chat can't overflow.
- **Errors**: at startup, a one-token call catches a bad key. In the loop, wrap `handle_message()` in `try/except` for `openai.AuthenticationError` (exit), `openai.RateLimitError` (one line, keep going), and `openai.BadRequestError` (drop the last message, ask to shorten). A missing key is caught separately with `MissingConfigError` before the loop.

Still stuck? The whole thing, every file, is in the complete solution.